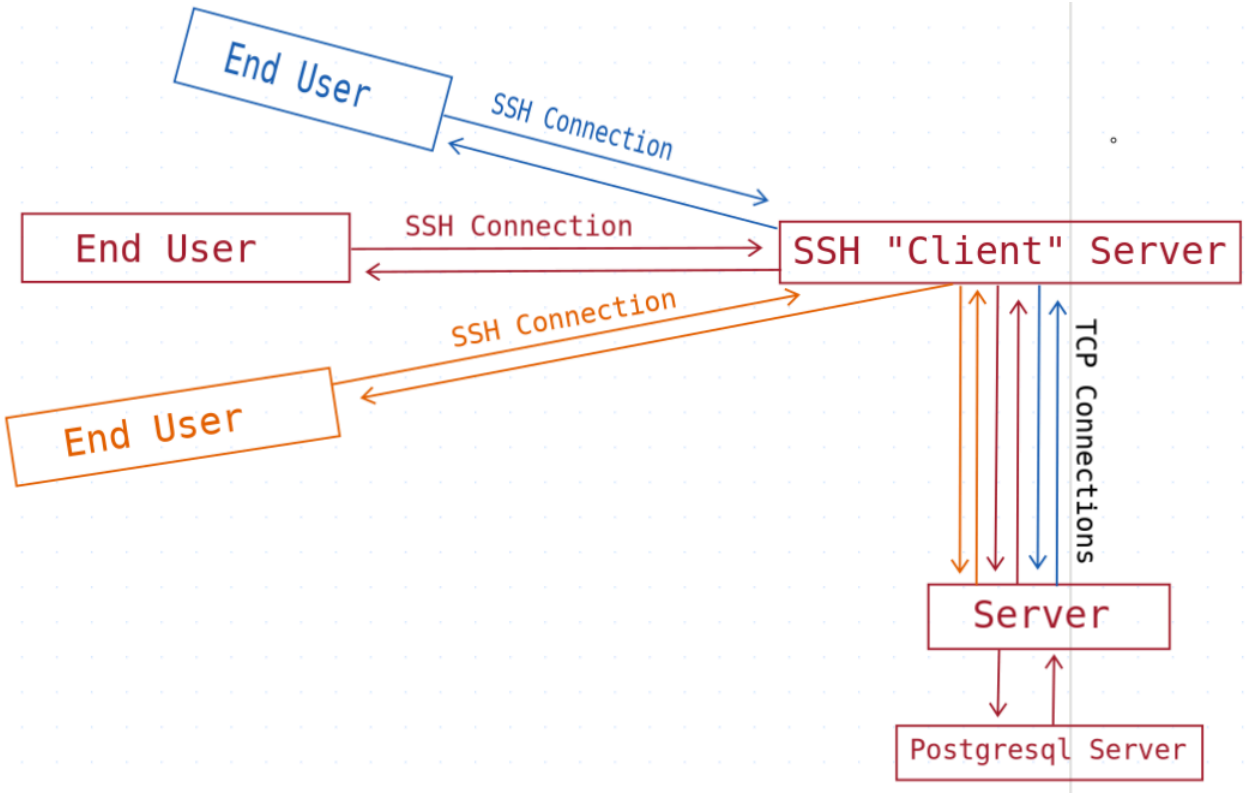
Initial Connection

The initial connection takes part in two separate phases. In the first phase, the end user establishes an SSH connection to the “client” server. In the second phase, the “client” server then establishes a TCP connection to the backend server. For the SSH connection portion, I rely on a library that implements the protocol for me. This is for a variety of reasons, but mainly because there are major security concerns in implementing such a system myself and I just didn’t have enough time to try implementing the protocol myself.

General Connection Schema

The general connection scheme looks as such:



A variable number of end users connect to a single SSH server. This SSH server is the machine that runs the actual client code, ie, processes user input, renders the application, updates ui state, etc. All the end user’s machine does is display the output rendered by the SSH server and send raw keystrokes to it. In essence, the SSH server acts as a middleman.

The SSH server makes a new TCP connection to the backend server for every end user that connects to it. Admittedly, this could be simplified to only be a single connection, but that would require me to restructure a good chunk of the project, which I don’t have the time to do as of now. The SSH server processes the raw keystroke data that comes in from the end users and turns that into application inputs. The application then takes those inputs and sends packets to the backend server to update its state.

Finally, the backend server connects to the Postgresql server, which lives on the same machine as the server.

Client Server Communication

My server and client communicate using binary encoded packets sent over a TCP socket. The packet has a basic uniform structure, which is represented in hexadecimal as:

0x0000000000000000000…

[2 Bytes]

The first two bytes of the binary encoded message always identifies the PacketType, an encoded short. This allows the PacketDecoder to identify what type of packet is being sent and defer to the proper decoder to reconstruct the encoded data. The decoding of this section is handled directly by the PacketDecoder class. The information returned from this section is then used by the PacketDecoder to construct the specific packet that will be used to further decode the incoming data.

[8 Bytes]

The next eight bytes of the message identifies the message id, an encoded ulong. The server reads this and attaches the same message id as the packet it is responding to to its response. This allows the client to identify which packet is responding to the message it sent out. This message is a ulong to ensure that ids will not be exhausted. This system can and should be improved, reusing dead ids can both reduce the amount of data that I need to send over the network, I could trim it down to 2 bytes, and eliminate the possibility of exhausting all packet ids, unless there are 30k+ packets being waited on simultaneously. Ids are also not needed for all packets, so always sending it is wasting 8 bytes. The information here is decoded by the AbstractPacket base class, ie, its shared, common functionality between all packets.

[Variable]

This section depends on the message being sent, some will have no extra information, such as the ViewLeaderboard packet. Others, such as the JoinLobby packet, include an extra 4 bytes for the integer representing the lobby id. The information here is decoded by the specific packet implementation.

Packets

Upon the receipt of any malformed packets, the server reply with a Denied packet, and then send a CloseConnection packet and close the connection.

Initial Handshake

OpenConnection

Size: 10 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long)

Recipient: Server
Usage: Sent by the client to notify the server that it wants to open a connection. If the server doesn't respond, assume that it is down.
Expected Response: Accepted, no inner packet

Login
Size: 106 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + Username (32 Bytes, string) + Password (64 Bytes, string) [Maybe make variable?]
Recipient: Server
Usage: Sent by the client to attempt a login, server responds with Accepted if username and password exist and match, responds with Denied if not.
Expected Response: Accepted, with inner LoginInformation packet, if successful, Denied if unsuccessful

Register
Size: 106 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + Username (32 Bytes, string) + Password (64 Bytes, string) [Maybe make variable?]
Recipient: Server
Usage: Sent by the client to attempt to register a new user, server responds with Accepted if username isn't already taken, responds with Denied if it is.
Expected Response: Accepted, with inner LoginInformation packet, if successful, Denied if unsuccessful

LoginInformation
Size: 106 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + SessionId (64 Bytes, string)
Recipient: Client
Usage: Sent by the server to the client, wrapped in an Accepted packet, in response to a successful login or register attempt.
Expected Response: None

Lobby Management

CreateLobby
Size: 74 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + SessionId (64 Bytes, string)
Recipient: Server
Usage: Sent by the client to notify the server that it wants to create a lobby. The server either responds with Accepted if the user isn't already in a lobby, or Denied if they are.
Expected Response: Accepted, with inner LobbyCreationInformation packet, if successful, Denied if unsuccessful

JoinLobby
Size: 78 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + SessionId (64 Bytes, string) + LobbyId (4 Bytes, int)

Recipient: Server
Usage: Sent by the client to notify the server that it wants to join a lobby. The server either responds with Accepted if the user isn't already in a lobby and the lobby exists, or Denied otherwise.
Expected Response: Accepted, with inner LobbyInformation packet, if successful, Denied if unsuccessful

LeaveLobby
Size: 74 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + SessionId (64 Bytes, string)
Recipient: Server
Usage: Sent by the client to notify the server that it wants to leave a lobby. The server either responds with Accepted if the user is in a lobby, or Denied otherwise. The server should notify the other user in the lobby that this user has left. It should default to giving the user that is still in the lobby the win if a game is ongoing.
Expected Response: Accepted, if successful, Denied if unsuccessful

LobbyCreationInformation
Size: 14 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + LobbyId (4 Bytes, int)
Recipient: Client
Usage: Sent by the server to the client to provide information about the lobby they just created. The server should create a unique lobby id that can be used to join this lobby and notify the client using this packet.
Expected Response: None

LobbyJoinInformation
Size: 42 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + Username (32 Bytes, int)
Recipient: Client
Usage: Sent by the server to the client to provide information about the lobby they just joined. The username field should be populated by the username of the other user in the lobby.
Expected Response: None

OpponentConnected
Size: 42 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + Username (32 Bytes, int)
Recipient: Client
Usage: Sent by the server to the client to notify them that an opponent has joined their lobby.
Expected Response: None

OpponentDisconnected
Size: 10 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long)
Recipient: Client
Usage: Sent by the server to the client to notify them that their opponent left the lobby. This should be sent both when the opponent explicitly leaves by sending a LeaveLobby packet and

when they implicitly leave by either sending a CloseConnection packet or failing to send a Heartbeat packet on time.
Expected Response: None

Game Management

BoardPlayed

Size: 78 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + SessionId (64 Bytes, string) + X (2 Bytes, short) + Y (2 Bytes, short)
Recipient: Server
Usage: Sent by the client to notify that it wants to place its mark on the specified spot on the board, indicated by the X and Y values. The server should respond with an Accepted packet if the client is in a game, it is the client's turn, and the position exists and is open, otherwise it should send a Denied packet. The server should also send an UpdateBoard packet to the other client in the lobby in the case that this packet is accepted.
Expected Response: Accepted, no inner packet, if successful, Denied if unsuccessful

UpdateBoard

Size: 14 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + X (2 Bytes, short) + Y (2 Bytes, short)
Recipient: Client
Usage: Sent by the server to notify the client in a lobby that the other player has played at a certain X and Y coord on the board.
Expected Response: None

GameCompleted

Size: 10 Bytes + 1 Bit | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + Won (1 bit, boolean)
Recipient: Client
Usage: Sent by the server to notify the client that the game it is currently in has completed. The Won bit indicates whether or not the client won the match.
Expected Response: None

Leaderboard

ViewLeaderboard

Size: 74 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + SessionId (64 Bytes, string)
Recipient: Server
Usage: Send by the client to notify the server that it wants to view the leaderboard.
Expected Response: Accepted, with inner LeaderboardInformation packet

LeaderboardInformation

Size: 198 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + …
Recipient: Client

Usage: Sent by the server in response to a ViewLeaderboard packet. Contains the usernames and win counts of the top 5 players, ordered by ranking. Also contains the position and number of wins of the requesting player.
Expected Response: None

General

Accepted

Size: Variable, Minimum 10 Bytes + 1 Bit | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + HasInnerPacket (1 Bit, boolean) + Inner Packet
Recipient: Client
Usage: Sent by the server to notify the client that its packet was received and accepted. If the HasInnerPacket field is 1, there is an inner packet that the client should decode for further information.
Expected Response: None

Denied

Size: 10 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long)
Recipient: Client
Usage: Sent by the server to notify the client that its packet was received and denied.
Expected Response: None

CloseConnection

Size: 74 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + SessionId (64 Bytes, string)
Recipient: Server, Client
Usage: Sent by either the server or the client to notify the other that it is disconnecting. If received by the client, this means that the server has forcefully disconnected it. If received by the server, this means that the client is leaving.
Expected Response: None, handle cleanly closing the connection

Heartbeat

Size: 10 Bytes | Packet Type (2 Bytes, short) + Id (8 Bytes, long) + SessionId (64 Bytes, string)
Recipient: Server
Usage: Sent by the client to the connected server to ensure that the connection is still alive and well. If no response is received, assume that the connection is stale and close it. If no heartbeat is received on the server for over 10 seconds, the server will assume that the client connection is stale and disconnect from it.
Expected Response: Accepted, no inner packet